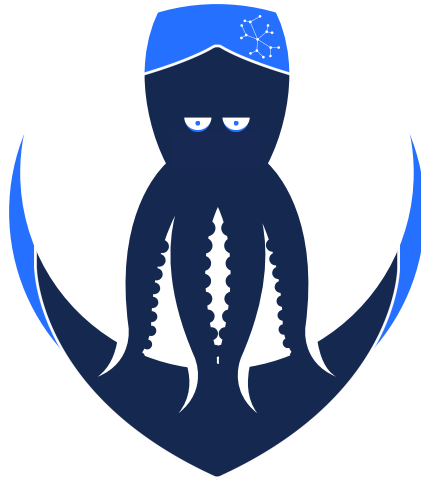




# Protocol Audit Report

Version 1.0

*Independent Security Review*

September 13, 2026

# PasswordStore Audit Report

MD AIMAN

August 18, 2026

Prepared by: MD AIMAN Lead Auditors: - MD Aiman

## Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - \* [H-1] Storing the password on-chain makes it visible to anyone, and no longer private
  - Likelihood & Impact:
    - \* [H-2] `PasswordStore::setPassword` has no access control, meaning a non-owner could change the password.
  - Likelihood & Impact:
  - Informational

\* [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

- Likelihood & Impact:
- Gas

## Protocol Summary

PasswordStore is a protocol dedicated to the storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

## Disclaimer

MD Aiman makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibility for the findings provided in this document. A security review is not an endorsement of the underlying business or product. The review was time-boxed, and the assessment of the code was solely on the security aspects of the Solidity implementation of the contracts.

## Risk Classification

|            |        | Impact |        |     |
|------------|--------|--------|--------|-----|
|            |        | High   | Medium | Low |
| Likelihood | High   | H      | H/M    | M   |
|            | Medium | H/M    | M      | M/L |
|            | Low    | M      | M/L    | L   |

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

## Audit Details

The findings described in this document correspond to the following commit hash:

```
1 <!-- TODO: paste the actual commit hash you audited, e.g. run `git log  
-1 --format="%H"` in the project repo -->
```

## Scope

```
1 ./src/  
2 #-- PasswordStore.sol
```

## Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

## Executive Summary

This review was conducted as an independent security assessment of the PasswordStore protocol, focusing on the single in-scope contract. The methodology combined manual code review with targeted proof-of-concept testing to validate exploitability of identified issues.

Two high-severity issues were identified, both stemming from a mismatch between the contract's intended access model and its actual implementation: private data stored on-chain is not actually private, and a state-changing function intended to be owner-only lacks any access control. One informational issue relating to incorrect NatSpec documentation was also noted.

## Issues found

| Severity | Number of issues found |
|----------|------------------------|
| High     | 2                      |
| Medium   | 0                      |
| Low      | 0                      |

| Severity | Number of issues found |
|----------|------------------------|
| Info     | 1                      |
| Total    | 3                      |

## Findings

### High

#### [H-1] Storing the password on-chain makes it visible to anyone, and no longer private

**Description:** All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore : : s_password` variable is intended to be a private variable and only accessed through the `PasswordStore : : getPassword` function, which is intended to be callable only by the owner of the contract.

We show one such method of reading any data off chain below.

**Impact** Anyone can read the private password, severely breaking the functionality of the protocol.

**Proof of Concept:** (Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
1 make anvil
```

2. Deploy the contract to the chain

```
1 make deploy
```

3. Run the storage tool

We use 1 because that's the storage slot of `s_password` in the contract.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this:

[illegible]

You can then parse that hex to a string with:

[illegible]

And get an output of:

```
1 myPassword
```

**Recommended Mitigation:** Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function, as you wouldn't want the user to accidentally send a transaction with the password that decrypts their password.

### Likelihood & Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

**[H-2] PasswordStore::setPassword has no access control, meaning a non-owner could change the password.**

**Description:** The `PasswordStore : : setPassword` function is set to be an `external` function, however, the `NatSpec` of the function and overall purpose of the smart contract is that `The function allows only the owner to set a new password.`

```
1 function setPassword(string memory newPassword) external {
2   @>    // @audit - There are no access controls
3         s_password = newPassword;
4         emit SetNetPassword();
5     }
```

**Impact:** Anyone can set/change the password of the contract, severely breaking the contract's intended functionality.

**Proof of Concept:** Add the following to the `PasswordStore.t.sol` test file.

## Code

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3     vm.prank(randomAddress);
4     string memory expectedPassword = "myNewPassword";
5     passwordStore.setPassword(expectedPassword);
6
7     vm.prank(owner);
8     string memory actualPassword = passwordStore.getPassword();
9     assertEq(actualPassword, expectedPassword);
10 }
```

**Recommended Mitigation:** Add an access control conditional to the `setPassword` function

```
1 if(msg.sender != s_owner){
2     revert PasswordStore__NotOwner();
3 }
```

## Likelihood & Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

## Informational

**[I-1] The PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.**

### Description:

```
1 ```javascript
2 /*
3  * @notice This allows only the owner to retrieve the password.
4  * @param newPassword The new password to set.
5  */
6 function getPassword() external view returns (string memory) {}
7 ```
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the NatSpec says it should be `getPassword(string)`. **Impact:** The NatSpec is incorrect. **Recommended Mitigation:** Remove the incorrect NatSpec line.

```
1 - * @param newPassword The new password to set.
```

**Likelihood & Impact:**

- Impact: NONE
- Likelihood: HIGH
- Severity: Informational/Gas/Non-Crits

Informational: Hey, this isn't a risk, but you should know...

**Gas**

No gas-related findings were identified in this review.